



KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT
DEPARTMENT OF COMPUTER AND ELECTRONICS COMMUNICATION &
INFORMATION ENGINEERING

A
MAJOR PROJECT REPORT
ON
“Road Condition Detection using YOLOv8 with E-mail Alert System”

By:

Bhuwan Adhikari (KIC078BEI007)
Bibek Khanal (KIC078BEI008)
Ankit Mandal (KIC078BEI009)

Lalitpur, Nepal

March, 2026



KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT
BALKUMARI, LALITPUR

“Road Condition Detection using YOLOv8 with E-mail Alert System”
EX755

By:
Bhuwan Adhikari (07/BEI/2021)
Bibek Khanal (08/BEI/2021)
Ankit Mandal (09/BEI/2021)

A PROJECT WAS SUBMITTED TO THE DEPARTMENT OF COMPUTER AND
ELECTRONICS, COMMUNICATION & INFORMATION ENGINEERING IN
PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE BACHELOR’S
DEGREE IN ELECTRONICS, COMMUNICATION & INFORMATION
ENGINEERING

DEPARTMENT OF COMPUTER AND ELECTRONICS, COMMUNICATION &
INFORMATION ENGINEERING
LALITPUR, NEPAL

March, 2026

KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT

BALKUMARI, LALITPUR

DEPARTMENT OF COMPUTER AND ELECTRONICS, COMMUNICATION &
INFORMATION ENGINEERING

The undersigned certify that they have read, and recommended to this department for acceptance, a project report entitled " **ROAD CONDITION DETECTION USING YOLOV8 WITH E-MAIL ALERT SYSTEM** " submitted by **ANKIT MANDAL, BIBEK KHANAL AND BHUWAN ADHIKARI** in partial fulfilment of the requirements for the Bachelor's degree in Electronics & Communication.

Supervisor, Er. Chaitya Shova Shakya
HOD
Department of Computer and Electronics

External Examiner, Er. Smriti Nakarmi
DHOD
KEC, Kalimati

Coordinator, Er.Saban Kumar K.C.
Assistant Professor
Department of Computer and Electronics

DATE OF APPROVAL:

COPYRIGHT

The author has agreed that the Library, Department of Computer and Electronics, Communication and Information Engineering, Kathford International College may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisors who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that the recognition will be given to the author of this report and to the Department of Computer and Electronics, Communication and Information Engineering, Kathford International College in any use of the material of this project report. Copying or publication or the other use of this report for financial gain without approval of to the Department of Computer and Electronics, Communication and Information Engineering, Kathford International College and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Computer and Electronics, Communication and Information Engineering,
Kathford International College of Engineering and Management,

Lalitpur, Nepal

DEPARTMENTAL ACCEPTANCE LETTER

DEPARTMENTAL ACCEPTANCE

The project entitled “**Road Condition Detection using YOLOv8 with E-mail Alert System**”, submitted by Ankit Mandal, Bibek Khanal and Bhuwan Adhikari in partial fulfilment of the requirements for the Bachelor’s degree in Electronics, Communication & Information / Computer Engineering has been accepted as a bonafide record of work carried out by them in this department.

Chaitya Shova Shakya

HOD

Department of Computer and Electronics, Communication and Information

Engineering, Kathford International College of Engineering and Management,

Lalitpur, Nepal

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have supported and guided us throughout the completion of our major project, **“Road Condition Detection using YOLOv8 with E-mail Alert System.”**

Firstly, we extend our sincere thanks to our supervisor, **Er. Chaitya Shova Shakya**, for her invaluable guidance, continuous support, and assistance in selecting this project title. Her insights and encouragement have been instrumental in shaping and successfully completing this project. We would also like to express our sincere gratitude to **Er. Saban Kumar K.C**, Project Coordinator, for his valuable guidance, coordination, and support throughout the project. His suggestions and supervision have greatly contributed to the smooth execution of our work.

Furthermore, we extend our appreciation to all the respected faculty members of the **Department of Computer and Electronics Communication & Information Engineering** for their continuous encouragement, academic guidance, and for providing us with the necessary resources to complete this project successfully.

We would also like to acknowledge **Robokath**, the Robotics Club of Kathford College, for their encouragement and collaboration. Additionally, we extend our gratitude to our peers and fellow students for their constant support and motivation throughout the project. Their valuable suggestions and insights have significantly enriched our work.

Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project.

Thank you all for being a part of this endeavor.

Ankit Mandal

Bibek Khanal

Bhuwan Adhikari

ABSTRACT

Road infrastructure plays a vital role in ensuring safe and efficient transportation. However, road surfaces deteriorate over time due to heavy traffic loads, environmental factors, and delayed maintenance. These leads to defects such as potholes and surface erosion on the road. Failure to detect these defects in a timely manner can result in accidents, vehicle damage, and increased maintenance costs. This project developed an intelligent road condition monitoring system titled “Road Condition Detection using YOLOv8 with Email Alert System.” The system uses the YOLOv8 deep learning model to automatically detect road surface defects such as potholes from images captured by a camera module. It has implemented a Raspberry Pi 5 to enable real-time image processing and inference using a trained YOLOv8n model. Upon detecting a defect, the system captures the geographic location using a GPS module and sends an automated email notification that contains the image, timestamp, and coordinates of that location to the relevant authorities for prompt action. Additionally, all detection data is stored in an external storage for further analysis and documentation. The model was trained on a dataset of 9,273 labeled images sourced from Kaggle, and demonstrated strong performance in terms of precision (85.6%), recall (87.7%), and mean Average Precision (mAP, mAP50: 0.71, mAP50-90: 0.64) .

A mobile rover platform provided experimental validation, confirming the system’s capability for real-time deployment. Overall, the system offers a cost-effective, scalable, and automated solution for road condition monitoring, allowing timely infrastructure maintenance for improved road safety.

Keywords: *YOLOv8, Deep Learning, potholes, E-mail Alert system*

TABLE OF CONTENTS

LETTER OF APPROVAL	i
COPYRIGHT	ii
DEPARTMENTAL ACCEPTANCE LETTER	iii
ABSTRACT.....	v
TABLE OF CONTENTS	vi
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
1. INTRODUCTION.....	1
1.1 Background.....	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope.....	3
2. LITERATURE REVIEW	4
2.1 Review of existing system	4
2.2 Related theory	6
3. PROJECT METHODOLOGY.....	13
3.1 Block Diagram of the System.....	13
3.2 Flowchart of the System	16
3.3 Dataset Description.....	17
3.4 System Implementation	21
4. HARDWARE AND SOFTWARE TOOLS.....	22

4.1	Hardware Tools.....	22
4.2	Software Requirements	24
5.	RESULT AND DISCUSSION	26
6.	CONCLUSION	35
7.	LIMITATIONS AND FUTURE WORK.....	36
7.1	Limitations	36
7.2	Future Work	37
8.	REFERENCES.....	38
	APPENDICES	39

LIST OF FIGURES

<i>Figure 2.1: YoloV8 Architecture.....</i>	8
<i>Figure 2.2: YoloV8 Backbone Section</i>	9
<i>Figure 2.3: YoloV8 Neck and Head Section</i>	11
<i>Figure 3.1: Block Diagram of the system</i>	13
<i>Figure 3.2: Flowchart of the system</i>	16
<i>Figure 3.3: Dataset visualization</i>	18
<i>Figure 5.1: Confusion matrix</i>	26
<i>Figure 5.2: F1-Confidence Curve.....</i>	27
<i>Figure 5.3: Precision-Confidence Curve.....</i>	28
<i>Figure 5.4: Precision-Recall Curve.....</i>	28
<i>Figure 5.5: Recall-Confidence.....</i>	29
<i>Figure 5.6: Performance metrics.....</i>	29
<i>Figure 5.7: Potholes detected by the model.....</i>	31
<i>Figure 5.8: Prototype of rover implementation</i>	32
<i>Figure 5.9: Email alert generated by the system</i>	33

LIST OF TABLES

<i>Table 2.1: YoloV8 variants</i>	6
<i>Table 3.1: Dataset description.....</i>	17
<i>Table 5.1: Confusion matrix values</i>	26
<i>Table 5.2: mAP metrics.....</i>	30
<i>Table 5.3: Error Analysis</i>	31

LIST OF ABBREVIATIONS

YOLO: You Only Look Once

IoU: Intersection over Union

DNN: Deep Neural Network

SMTP: Simple Mail Transfer Protocol

API: Application Programming Interface

CNN: Convolutional Neural Network

NMS: Non-Maximum Suppression

SPPF: Spatial Pyramid Pooling Fast

C2F: Coordinates-To-Features

ONNX: Open Neural Network Exchange

1. INTRODUCTION

1.1 Background

Road infrastructure is very crucial in ensuring safe and efficient transportation, as well as supporting economic development. However, road surfaces are frequently subjected to deterioration due to heavy traffic loads, environmental conditions, and inadequate maintenance. This leads to defects such as potholes and surface degradation, which not only reduce driving comfort but also increase the risk of accidents and vehicle damage if not detected and maintained immediately.

Traditionally, road condition monitoring has been done by manual inspections and basic surveillance systems. These approaches are time-consuming and labor-intensive. Also, human inspections may overlook minor defects or produce inconsistent assessments. This results in limited coverage and reduced reliability. Existing automated solutions run on sensor-mounted vehicles like accelerators or gyroscopes to detect vibrations due to irregularities on the roads. However, these systems usually produce inaccurate results because they cannot distinguish between different types of road defects and also affected by vehicle speed or smoothness [1]. Similarly, conventional image processing methods can hardly respond to road damages under varying lighting conditions, weather changes or complex road textures [2]. With advancements in computer vision and deep learning, automated approaches become effective solutions for road surface defect detection. The YOLOv8 (You Only Look Once, version 8) model offers high accuracy and fast processing speed, making it suitable for real-world deployment [5].

Many road defects remain undetected until they become severe, and in some cases, they are not reported promptly to the relevant authorities, leading to delayed maintenance and higher repair costs. Therefore, an automated alert system is useful, where images captured along with GPS coordinates and timestamps are transmitted to the authorities for timely intervention. One of the most effective mechanisms for timely notification is an automated email alert system.

Raspberry Pi 5 is a low-cost, portable, and scalable platform that can run the entire system for real-time monitoring and can be deployed at various locations.

The general idea of this project is to develop an efficient, automated, and real-time road condition monitoring system that improves detection accuracy, reduces manual work, and enhances road safety.

1.2 Problem Statement

Road surface deterioration caused by heavy traffic, environmental conditions, and inadequate maintenance leads to defects such as potholes and cracks, which reduces road safety and increases vehicle damage. Many of these defects remain undetected until they become severe, and in some cases, are not reported promptly to the concerned authorities, resulting in delayed maintenance and increased repair costs.

Conventional methods based on manual inspection are time-consuming, labor-intensive, and often prone to human error. Existing automated approaches, including sensor-based and traditional image processing techniques, often lack accuracy and robustness under varying operating conditions.

Therefore, there is a need for an intelligent and reliable system capable of real-time accurate detection of road defects with minimal human intervention, along with prompt reporting to the concerned authorities.

1.3 Objectives

The objective of this project is to design and implement an automated, deep learning-based system using YOLOv8 for real-time detection and classification of road surface defects, integrated with an e-mail alert mechanism to promptly notify relevant authorities for timely maintenance and improved road safety.

- i. Implement and train a YOLOv8 model for accurate detection and classification of various road surface defects from images.

- ii. Integrate the detection system with an automated Email alert mechanism for real-time notifications.
- iii. To build and test a real-time road monitoring system using Raspberry Pi, camera, and GPS module for practical implementation.

1.4 Scope

The scope of this project includes the design and development of a working prototype that captures road images, detects surface defects in real time using deep learning techniques, and automatically notifies the relevant authorities.

The system integrates both hardware and software components, where a Raspberry Pi 5 serves as the processing unit, along with a camera module for image capture and a GPS module for location tracking. The road images are processed using the YOLOv8 object detection model to identify common defects such as potholes with high accuracy. Upon detecting defects, the system automatically generates email notification.

The system is limited to detecting visible surface defects and does not handle underground or structural damage. It can be deployed in smart transportation systems, municipal road monitoring, and infrastructure maintenance.

2. LITERATURE REVIEW

2.1 Review of existing system

The detection of road surface defects has evolved significantly with the advancement of deep learning and computer vision techniques. Traditional manual inspection methods, while effective to some extent, are labor-intensive, time-consuming, and prone to human error. The emergence of deep convolutional neural networks (CNNs) has enabled the automation of road condition assessment with higher accuracy and efficiency.

Kulambayev et al. [1] proposed a Mask R-CNN-based deep learning model for real-time identification and segmentation of road surface defects. Their approach involved collecting and annotating a dataset, training the model using the TensorFlow framework, and achieving high detection and segmentation performance. The Mask R-CNN model reported precision, recall, and F1-score values of 0.9214, 0.9876, and 0.9571, respectively, and an Intersection over Union (IoU) of 0.3488 for segmentation tasks. This work demonstrated the effectiveness of deep learning in automating road surface quality assessment, even with limited training data.

Other studies have explored various neural network architectures for road damage detection. For example, encoder–decoder networks and fully convolutional neural networks (FCNNs) have been employed to segment and classify potholes at the pixel level. VGG-16, a well-known CNN architecture, has also been adapted for road crack detection, showing high recognition quality even on previously unseen images.

Object detection frameworks, particularly the YOLO (You Only Look Once) family, have gained prominence due to their real-time processing capabilities and high detection accuracy. YOLO models, including the latest YOLOv8, are designed to detect multiple objects in a single forward pass, making them suitable for applications requiring rapid analysis of large image datasets, such as road surveillance. YOLOv8 introduces improvements in both speed and accuracy over its predecessors, which is crucial for real-time road condition monitoring and alert systems.

Traditional Sensor-Based Approaches: Early automated systems utilized sensors like accelerometers and gyroscopes mounted on vehicles to detect vibrations indicative of road irregularities. While cost-effective and capable of real-time data acquisition, these methods often struggled with differentiating between various types of anomalies and were sensitive to vehicle speed and suspension characteristics [1].

Image Processing and Computer Vision: The proliferation of digital cameras led to image-based techniques, where road surface images were analyzed using classical image processing algorithms (e.g., edge detection, thresholding) to identify defects. Although providing visual evidence, these methods were often hampered by environmental factors such as lighting variations, shadows, and water accumulation, leading to potential false positives or missed detections [2].

Machine Learning Integration: The incorporation of traditional Machine Learning (ML) algorithms, such as Support Vector Machines (SVM) and Random Forests, with sensor and image data marked a significant improvement. These models could learn patterns from labeled datasets to classify road conditions. However, they frequently required extensive feature engineering and struggled with the inherent complexity and variability of real-world road data [3].

Deep Learning, a powerful subset of ML, has revolutionized road condition detection by enabling models to automatically learn hierarchical features directly from raw data, thereby minimizing the need for manual feature extraction. Convolutional Neural Networks (CNNs) for Image Data: CNNs have become the predominant architecture for image-based road defect detection due to their exceptional performance in image recognition and pattern detection. Researchers have successfully trained CNNs to identify and classify various road anomalies (e.g., potholes, patches) from images captured by dashboard cameras or specialized imaging systems. **Object Detection Models:** Advanced CNN-based object detection models like YOLO (You Only Look Once), Faster R-CNN, and SSD have been widely adopted for real-time pothole detection. These models are capable of not only classifying defects but also precisely localizing them with bounding boxes, providing crucial information about their position and dimensions [4], [5].

2.2 Related theory

YOLOv8, developed by Ultralytics, is a state-of-the-art object detection model pushing the boundaries of speed, accuracy, and user-friendliness. The acronym YOLO, which stands for “You Only Look Once,” enhances the algorithm’s efficiency and real-time processing capabilities by simultaneously predicting all bounding boxes in a single network pass. In comparison, several other object detection techniques require the detection process to go through several phases or processes. The popular YOLOv5 architecture is expanded upon in YOLOv8, which offers enhancements in these areas. The primary distinction between the YOLOv8 model and its predecessor is the utilization of anchor-free detection, which expedites the Non-Maximum Suppression (NMS) post-processing. YOLOv8 can identify and locate objects in images and videos with impressive speed and precision and tackles tasks like image classification and instance segmentation.

YOLOv8 Variants

Table 2.1: YoloV8 variants

Model Variant	d(depth_multiple)	w(width_multiple)	mc(max_channels)
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
xl	1.00	1.25	512

All of these models belong to the YOLOv8 family, each variant offers different trade-offs between accuracy, speed, and model size. The variants are divided based on the difference in the value of the parameters like depth_multiple (d), width_multiple (w), and max_channel (mc).

depth_multiple(d):

depth_multiple parameters determines how many Bottleneck Blocks are used in the C2f block. This scales the number of layers in the network. A value less than 1 reduces the depth (fewer layers), making the model smaller and faster but potentially less accurate. Conversely, a value greater than 1 increases the depth (more layers), leading to a larger and potentially more accurate model but slower to run.

width_multiple (w):

This scales the number of channels in the convolutional layers. A value less than 1 thins the network (fewer channels), resulting in a smaller and faster model but potentially sacrificing some accuracy. On the other hand, a value greater than 1 widens the network (more channels), creating a larger and potentially more accurate model but requiring more processing power.

max_channels (mc):

This parameter sets an upper limit on the number of channels allowed in the network. It is a safety measure to prevent the model from becoming too wide (too many channels) especially when width_multiple is set high. This can help control the model size and prevent overfitting.

Types of YOLOv8:

n: smallest model, fastest inference but lowest accuracy

s: small model, good balance of speed and accuracy

m: medium model, higher accuracy than small models with moderate inference speed

l: large model, highest accuracy but slowest inference

xl: extra-large model, best accuracy for resource-intensive applications

YOLOv8 Architecture

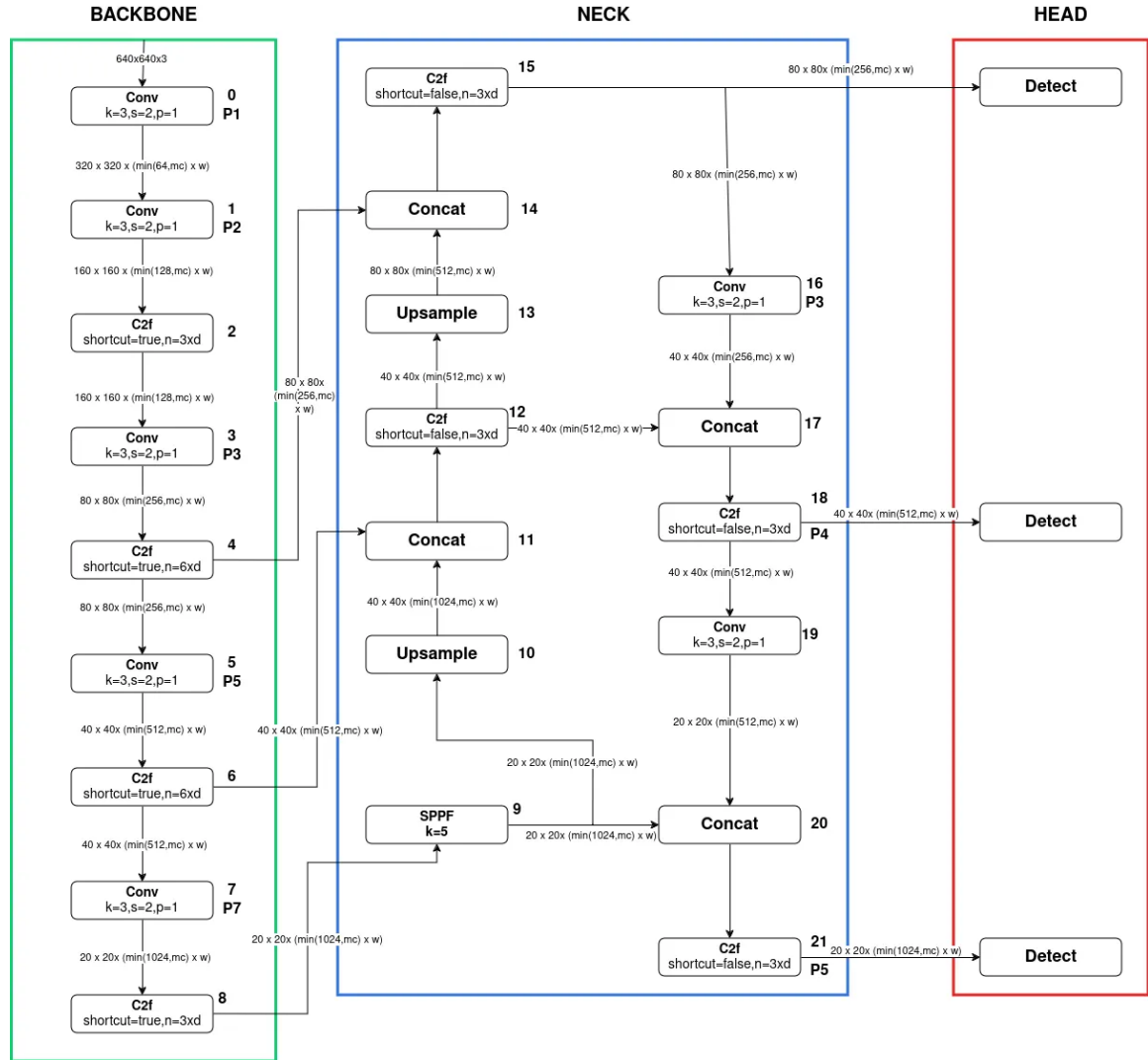


Figure 2.1: YoloV8 Architecture

YOLOv8 Architecture consists of three main sections: Backbone, Neck, and Head.

Backbone is the deep learning architecture that acts as a feature extractor of the inputted image. Neck combines the features acquired from the various layers of the Backbone module. Head predicts the classes and the bounding box of the objects which is the final output produced by the object detection model.

Backbone Section:

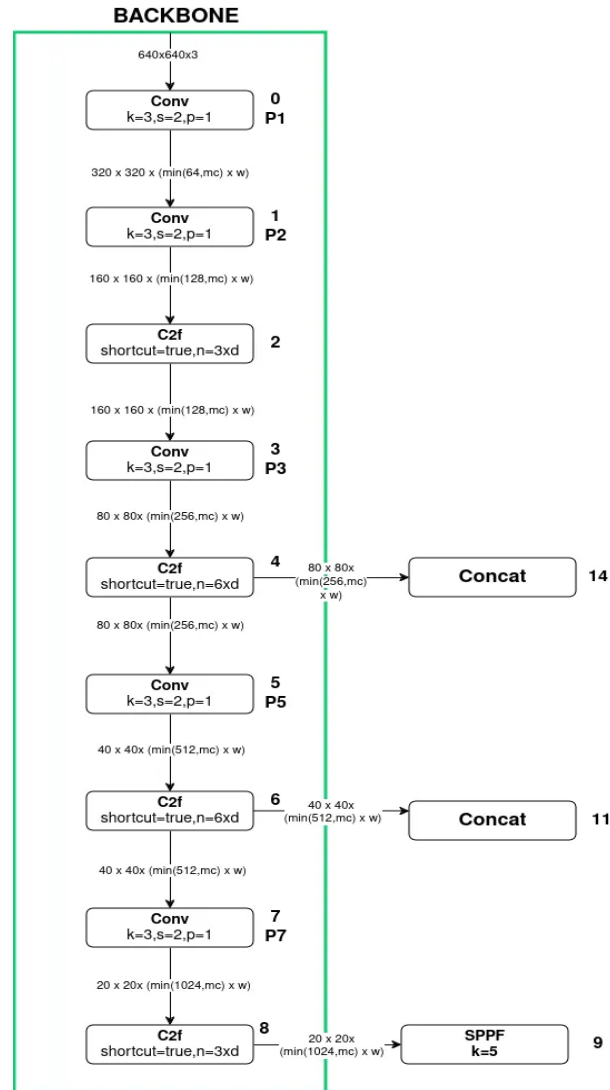


Figure 2.2: YoloV8 Backbone Section

In Block 0, the processing starts with the input image size of 640 x 640 x 3 which is fed to the convolutional block with kernel size 3, stride 2, and padding 1. The spatial resolution is reduced when stride= 2 is used. The convolutional block produces the feature map of 320 x 320 because the kernel moves in 2-pixel increments.

To obtain the output channel of the convolution block, the following formula is used :

$$\min(64,mc)*w$$

Here,

64 is the base output channel

mc is the max_channel

w is the width_multiple

For example, if we are using the “n” variant YOLOv8 model then our final output channel becomes $= \min(64, 1024) * 0.25 = 64 * 0.25 = 16$

Likewise, this operation is calculated in every convolutional block present in the architecture.

Block 2, is a C2f block that contains two parameters i.e. shortcut and n. Here, the shortcut is the boolean parameter that denotes if the Bottleneck block utilizes the shortcut or not. If the value of the shortcut= true then the bottleneck block inside the C2f block utilizes the shortcut else it doesn't.

Here, n determines how many bottleneck blocks are used inside the C2f block. In the case of Block 2, n is given by:

$$n = 3 * d$$

where d= depth_multiple

For example, if we are using the “n” variant YOLOv8 model the depth_multiple of the “n” type YOLOv8 model is 0.33 so,

the number of bottleneck block used inside the C2f becomes $(n) = 3 * 0.33$

$= 0.99$ i.e. 1 bottleneck block is used.

In the C2f block the resolution of the feature map and the output channel is unchanged.

In Block 9, the SPPF Block is used after the last convolution layer of the C2f block in the Backbone. The main function of the SPPF block is to generate the fixed feature

representation of the object in various sizes in an image without resizing the image or introducing spatial information loss.

Neck and Head Section

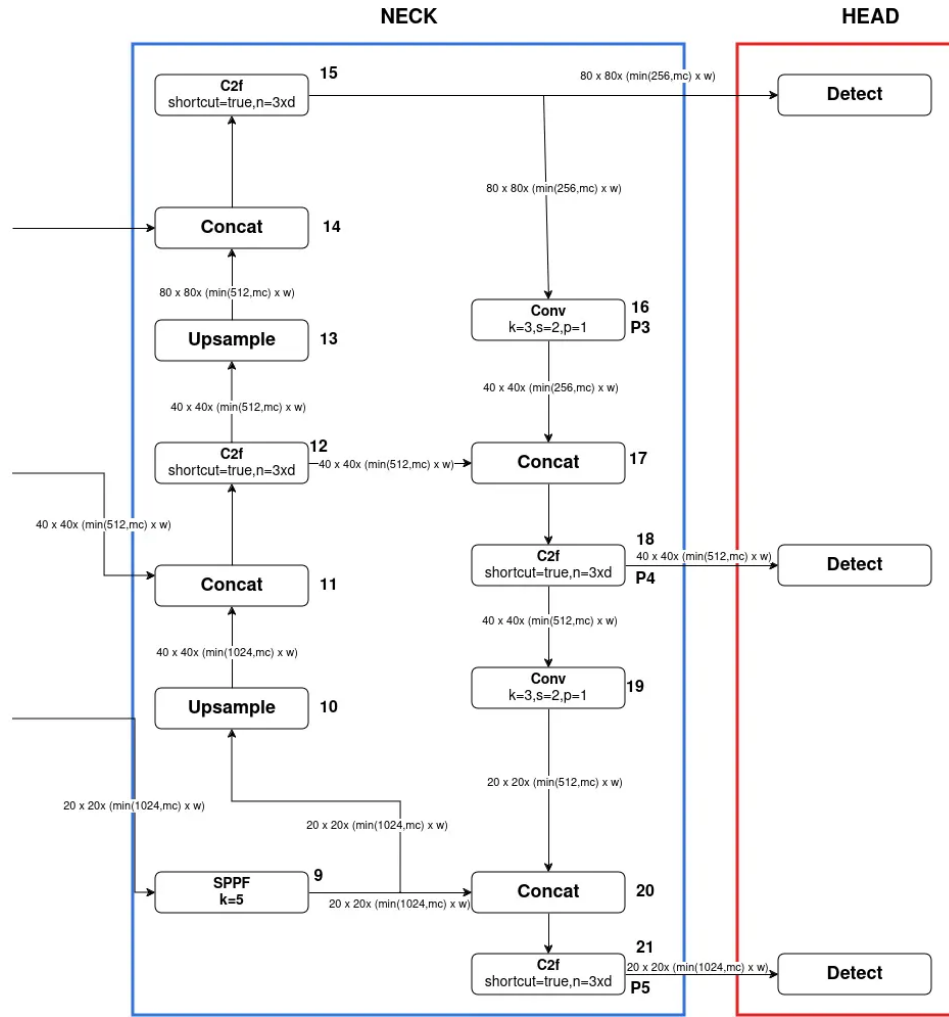


Figure 2.3: YoloV8 Neck and Head Section

The neck section is responsible for upsampling the feature map and combining the features acquired from the various layers of the Backbone section.

The upsample layer present in the Neck section simply increases the feature map by double without making any changes in the output channel.

Concat Block sums off the output channels of the blocks that are being concatenated without any change in resolution.

The head section is responsible for predicting the classes and the bounding box of the objects which is the final output produced by the object detection model.

The first Detect block in the Head section specializes in detecting small objects that are inputted from the C2f block present in Block 15.

The second Detect block in the Head section specializes in detecting medium-sized objects which is inputted from the C2f block present in Block 18.

The third Detect block in the Head section specializes in detecting small objects that are inputted from the C2f block present in Block 21.

3. PROJECT METHODOLOGY

3.1 Block Diagram of the System

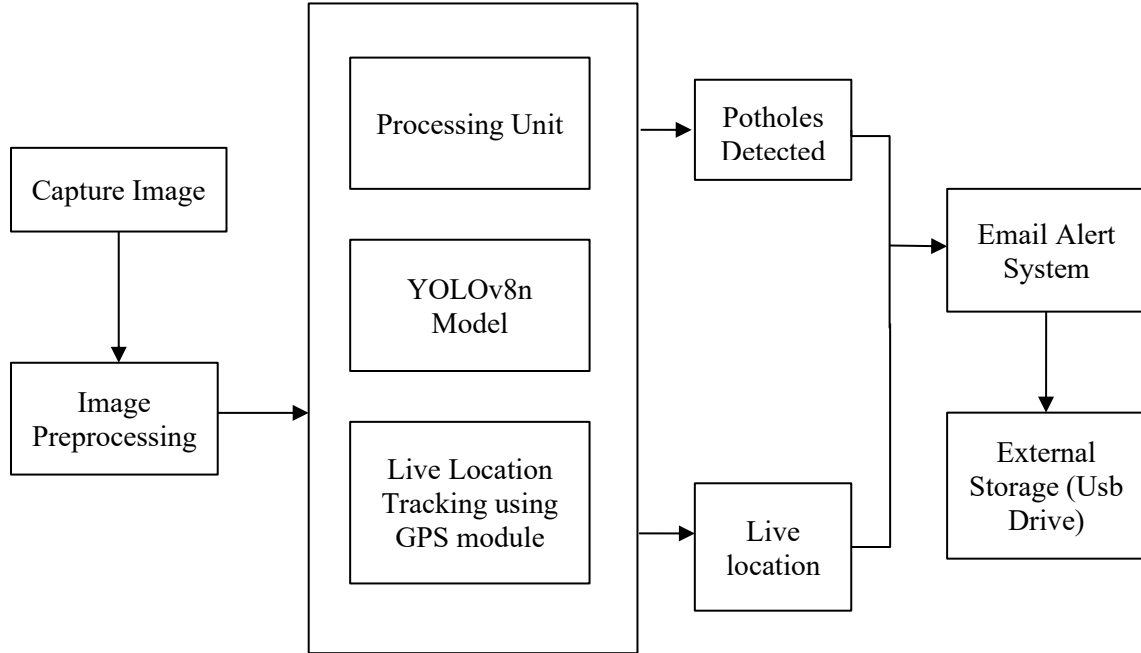


Figure 3.1: Block Diagram of the system

Working Principle of the System

This system can automatically detect road problems like potholes, and surface ware using a camera and a deep learning model. The process starts when a camera captures real-time images of the road surface. An image preprocessing unit prepares these images by adjusting their size and format so that the system can analyze them properly.

After preprocessing, the image is sent to the system's core, which runs on a Raspberry Pi 5. This small computer handles all operations. It uses a machine learning model called YOLOv8n that has been trained to recognize different types of road surface damage. The model checks the image and identifies any visible issues like potholes or surface wares.

If the model detects road damage, the system immediately collects the live GPS location using a connected GPS module. This location, along with the image and the time of

detection, is compiled into a report. The system then creates an email alert with all this information: an image of the damage, a clickable link to the GPS coordinates, and the timestamp.

This alert is automatically sent to a specific email address, such as the one used by the local road maintenance office or relevant authority. At the same time, the system saves a copy of the image and detection details on an external USB drive for future reference or evidence. The system continues to operate independently, constantly monitoring the road, detecting issues, recording data, and sending alerts without requiring direct supervision.

The system consists of several interconnected modules that work together to detect road surface defects and generate and send the alerts. Each module works as follows:

i. Image Capture (Pi Camera)

The kickoff point of the system is having live images of the roadbed captured by the Raspberry Pi camera. When the system is on, the camera is constantly capturing the road so that the road conditions can always be observed.

ii. Image Preprocessing

Images captured are processed and passed over to the detection model. Upsizing/downsizing of the image, sharpness, noise elimination and converting the picture to an ideal format all define this step. The power and precision of the detection model are enhanced through the preprocessing.

iii. Processing Unit (Raspberry Pi 5)

The Raspberry Pi 5 is the primary controlling part of the system. It handles all the operations, such as image processing, model of detection, gathering for the GPS system, and controlling the alert system. It ensures good coordination between the hardware and software components.

- iv. Detection Module YOLOv8 Algorithm
The preprocessed image is delivered to the YOLOv8 model that examines the picture and identifies road deformities (such as potholes) in the picture. It operates in real-time and the model draw bounding boxes around detected defects and classify them instantly.

- v. Live Position Tracking (GPS Module)
When a defect is detected, the GPS module obtains the actual geographic position (latitude and longitude) of the system. This aids in determining exact location where the road is damaged.

- vi. Email Alert System
The system automatically sends an email when the defect is detected. The email will contain the captured image of the defect, the GPS position (usually a map link), and the date and time of detection. This alert is delivered to the relevant authority to take timely action.

- vii. External Storage (USB Drive)
Any images and other related information (location, time of discovery) found are recorded in a USB. This enables referencing, analysis and account keeping of road conditions in the future.

- viii. Output (Detected Defects)
The ultimate product of the system is detection of road potholes. These are based on detections and are used in alerting and recording to maintain the records.

3.2 Flowchart of the System

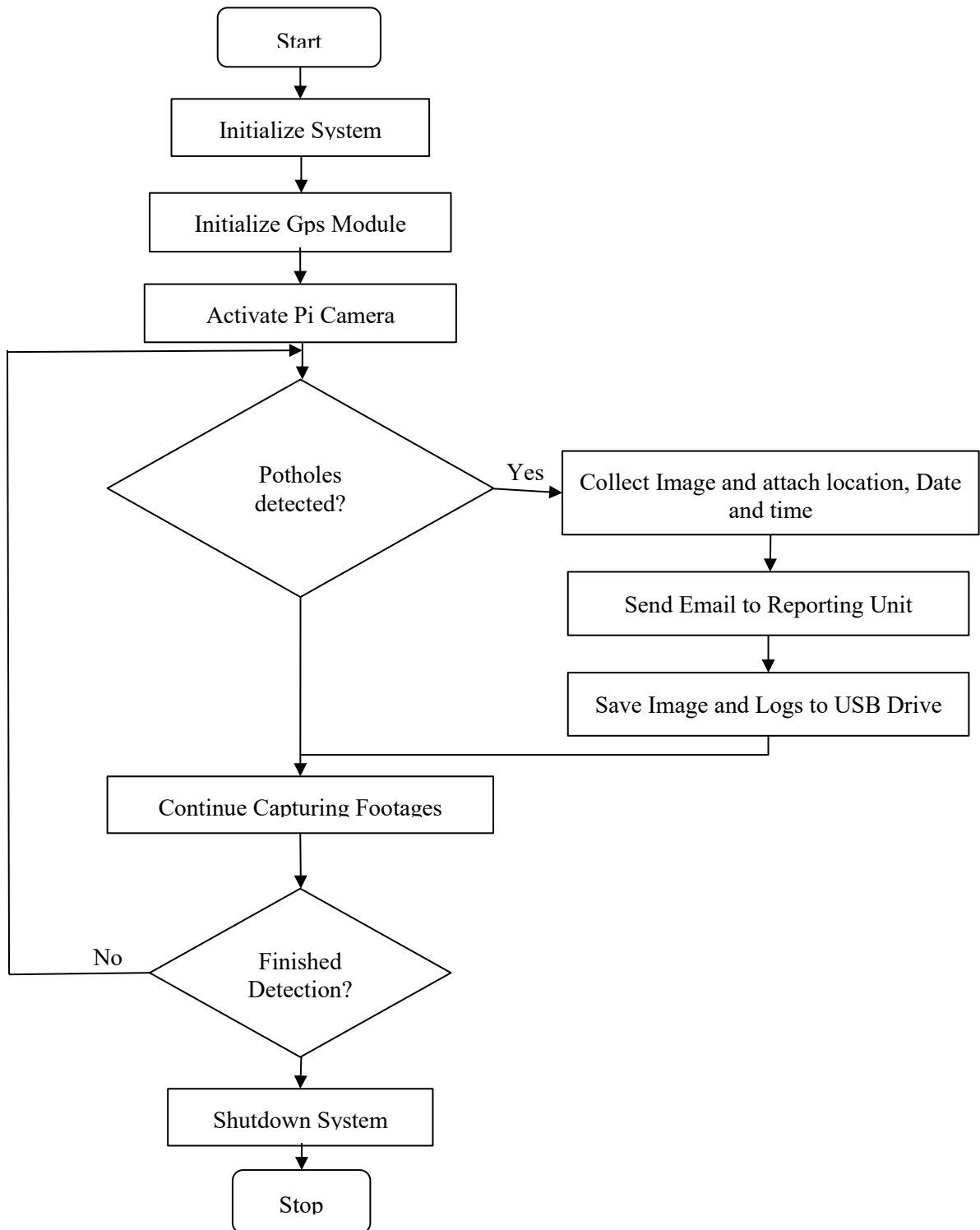


Figure 3.2: Flowchart of the system

3.3 Dataset Description

The dataset used for pothole detection consists of 9,273 images obtained from Kaggle, which were annotated in YOLO format for object detection. Each image contains labeled bounding boxes representing potholes.

Pre-processing Steps:

- Auto-orientation of images (removal of EXIF orientation)
- Resizing of all images to 640×640 pixels
- No data augmentation techniques were applied

Dataset Splits:

Table 3.1: Dataset description

Dataset Type	Number of Images
Training	7481
Validation	1219
Testing	573

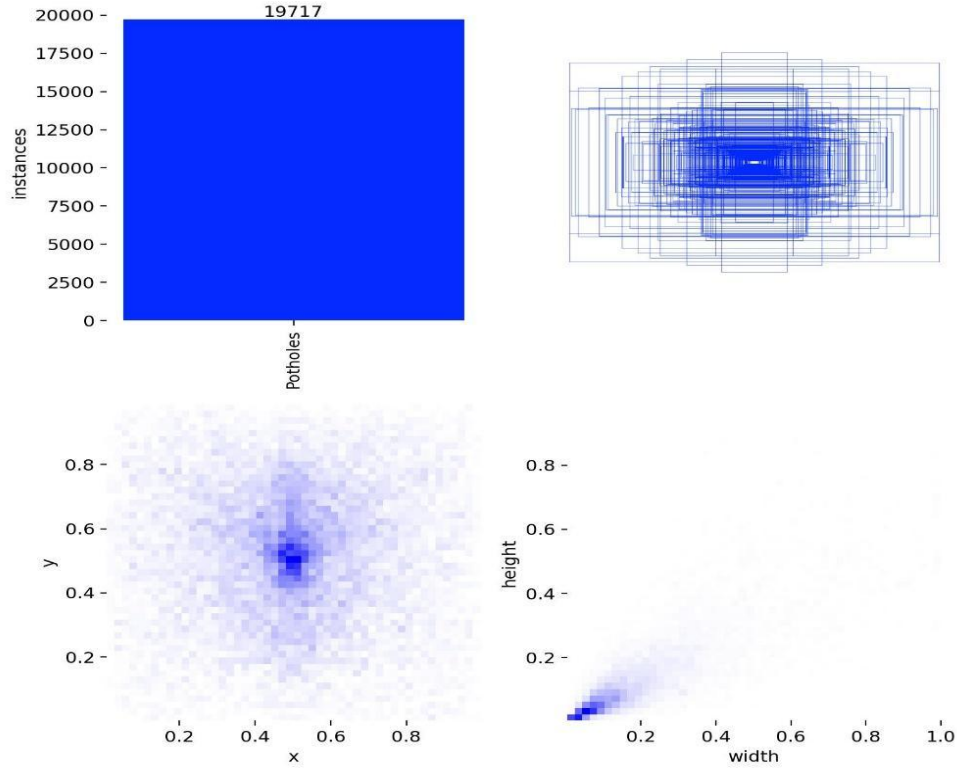


Figure 3.3: Dataset visualization

The label visualization provides insights into the distribution and characteristics of the annotated dataset used for pothole detection. The dataset contains approximately 19,717 labeled instances, indicating a sufficiently large dataset for training the model.

The bounding box visualization shows that most potholes are concentrated near the center of the images, revealing a positional bias in the dataset. This may affect the model’s ability to generalize well to objects located near the edges of the frame. The heatmap of object positions further confirms that the highest density of annotations is around the central region, with fewer samples at the boundaries. Additionally, the width and height distribution indicates that most potholes are small-sized objects, which makes detection more challenging and may lead to missed detections.

Overall, the dataset is well-populated but exhibits center bias and dominance of small objects, which can influence model performance. Improving dataset diversity in terms of object location and size can enhance detection accuracy.

Model Training:

The pothole detection model was developed using YOLOv8n, a lightweight and efficient object detection architecture suitable for real-time applications. The model was trained using the prepared dataset, where:

- Training data was used to learn pothole features
- Validation data was used to monitor performance and prevent overfitting

Training was carried out in 150 epochs which took 4.211 hours.

Model Evaluation Method:

The trained YOLOv8n model was evaluated using the testing dataset. The predictions were compared with ground truth annotations using the Intersection over Union (IoU) metric.

- A detection is considered correct if: $\text{IoU} \geq 0.5$
- Predicted class matches the ground truth

Performance Metrics

- **Precision**

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall**

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score**

$$F1 = \frac{2TP}{2TP + FP + FN}$$

- **Mean Average Precision (mAP)**

Mean Average Precision (mAP) is a standard metric used in object detection to evaluate both detection accuracy and localization performance.

mAP@0.5: Evaluates detections at IoU = 0.5

mAP@0.5:0.95: Evaluates across multiple IoU thresholds

3.4 System Implementation

4. HARDWARE AND SOFTWARE TOOLS

4.1 Hardware Tools

i. Raspberry Pi Model 5

The Raspberry Pi 5 is the latest in the line of single-board computers from the Raspberry Pi Foundation, which promises to offer a significant boost in performance compared to the previous models in the series. The Raspberry Pi 5 is powered by the Broadcom BCM2712 quad-core Arm Cortex-A76 CPU, which runs at 2.4Ghz, offering 2-3 times better CPU performance compared to the Raspberry Pi 4. It also comes with up to 16GB of LPDDR4X RAM, making it a powerful device for various purposes. It also features a VideoCore VII GPU, which runs at 800Mhz, offering support for OpenGL ES 3.1 and Vulkan 1.2, making it possible to display dual 4K@60fps on HDMI ports with HDR support. It also comes with a microSD card slot, as well as a PCIe 2.0 x1 interface, which can connect directly to NVMe SSDs, making it significantly faster compared to the previous models in the series.

The Raspberry Pi 5 is also well-equipped with connectivity features, including dual-band 802.11ac wireless LAN, Bluetooth 5.0 LE, Gigabit Ethernet with PoE+ support, two USB 3.0 ports, two USB 2.0 ports, as well as a 40-pin GPIO header, making it backward compatible with all the accessories designed for the previous models in the series. It also features a dedicated power button, which is a new feature in the Raspberry Pi 5. It runs on Raspberry Pi OS (64-bit) and needs to be powered by a 5V/5A USB-C supply for maximum performance.

ii. NEO-6M GPS Module

The NEO-6M is a high-performance GPS module with low costs, based on the u-blox NEO-6M chip. It is widely used in robotics, drones, and navigation applications. It can track 22 satellites and support 50 channels. It has an industry-leading tracking sensitivity of -161 dB. It also offers 2.5-meter position accuracy

and can provide 5 location updates per second. It also offers a Time-To-First-Fix (TTFF) of less than 1 second in hot start mode.

It also comes with an in-built 25 x 25 x 4mm ceramic active antenna, which offers strong satellite search capability. It also comes with power and signal LEDs, which can be used to monitor its status. In terms of hardware and communication, it works with 3-5V supply voltage. Its default baud rate is 9600. It also comes with an in-built EEPROM, a backup battery, and is compact in size, measuring 23mm x 30mm. It can communicate with microcontrollers via UART. Its baud rate can vary from 4800 bps to 230400 bps. It also comes with an in-built rechargeable battery. This battery retains clock and last position data. This reduces TTFF during power-up. It also comes with an in-built Power Save Mode (PSM) that reduces current consumption to 11 mA. It is also compatible with microcontrollers such as Raspberry Pi.

iii. Raspberry Pi Camera Module 3

The Raspberry Pi Camera Module 3 is the latest camera module announced by the Raspberry Pi Foundation. It is equipped with a significant upgrade from its previous model. It is equipped with a Sony IMX708 back-illuminated stacked CMOS 12-megapixel image sensor with a sensor size of 7.4mm diagonal and a pixel size of $1.4\mu\text{m} \times 1.4\mu\text{m}$. It provides a maximum resolution of 4608×2592 pixels. One of its most significant upgrades is that it is equipped with Phase Detection Autofocus (PDAF), making it the first Raspberry Pi camera with autofocus. It also provides HDR mode for richer quality. It is available in four versions: standard (75° field of view) and wide-angle (120° field of view), with or without an infrared cut filter for night vision.

For video capture, it provides standard video modes including 1080p at 50fps, 720p at 100fps, and 480p at 120fps. It captures images up to a maximum of 4608×2592 pixels. It provides CSI-2 serial data output. It is connected using a 200mm ribbon cable with a $15 \times 1\text{mm}$ FPC connector. It is compact with dimensions of $25 \times 24 \times$

11.5mm. It is fully supported by libcamera. It is compatible with all Raspberry Pi computers with a CSI connector.

iv. 16GB USB 3.0 Flash Drive

Saves images and detection outcomes as well as logs in an external storage where they will be accessed later.

v. Jumper Wires

Prob used to interface various hardware in the raspberry such as cameras and GPS module to the Raspberry Pi.

vi. 5V/3A Power Bank (5000–6000 mAh)

Provides energy to Raspberry Pi and components that are attached hence the system is portable.

4.2 Software Requirements

i. Python 3.10+

Principal language of programming on which the whole system was to be written and occasionally executed.

ii. Raspberry Pi OS

This is an operating system that was installed on the Raspberry Pi to control hardware and execute programs.

iii. Google Colab

Mainly used to train the YOLOv8 model which supports use with a graphic card (i.e. gpu).

iv. Ultralytics YOLOv8n

Small YOLOv8 model that is applicable in real-time in detecting the presence of defects on the road.

v. PyTorch with CUDA

Deep learning model to train and execute the YOLOv8 model effectively with the aid of GPU acceleration.

vi. OpenCV

Image processing library that is used in capturing, resizing and pre processing images.

vii. smtplib (for email)

Python library was to be used to send automatic email notification with alerts of detected defects.

viii. NMEA Parser for GPS Data

Digested the raw GPS information in the GPS module to process the location coordinates.

5. RESULT AND DISCUSSION

The performance of the road condition detection system was evaluated using the trained YOLOv8n model on the test dataset. The evaluation includes analysis of the confusion matrix, performance curves, and standard object detection metrics such as precision, recall, F1-score, and mean Average Precision (mAP).

Confusion Matrix:

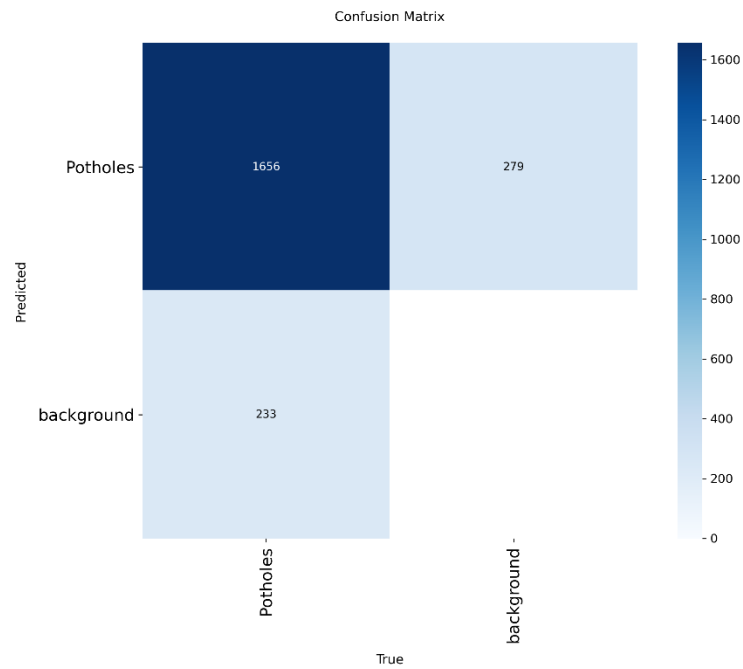


Figure 5.1: Confusion matrix

The confusion matrix is constructed based on object-level predictions:

Table 5.1: Confusion matrix values

	Actual Pothole	Actual Background
Predicted Pothole	1656	279
Predicted Background	233	0

Where:

- True Positive (TP) = 1656
- False Positive (FP) = 279
- False Negative (FN) = 233
- True Negative (TN) ≈ 0

True negatives are not explicitly defined in object detection tasks; therefore, accuracy is not considered a reliable metric.

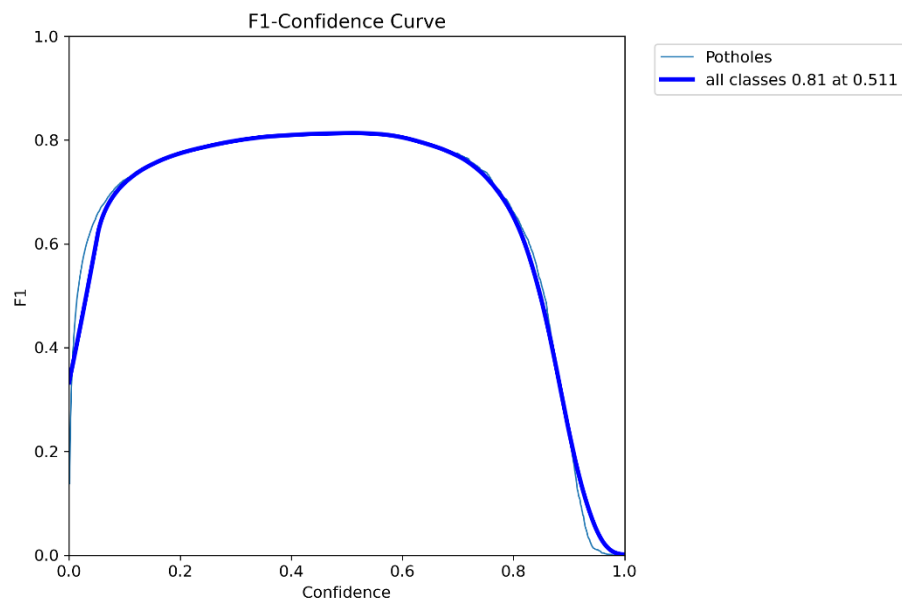


Figure 5.2: F1-Confidence Curve

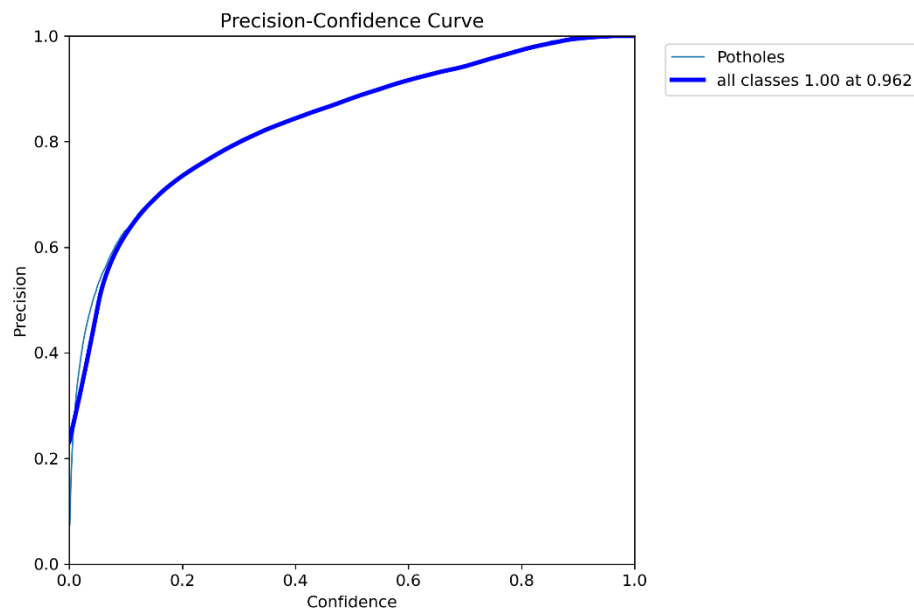


Figure 5.3: Precision-Confidence Curve

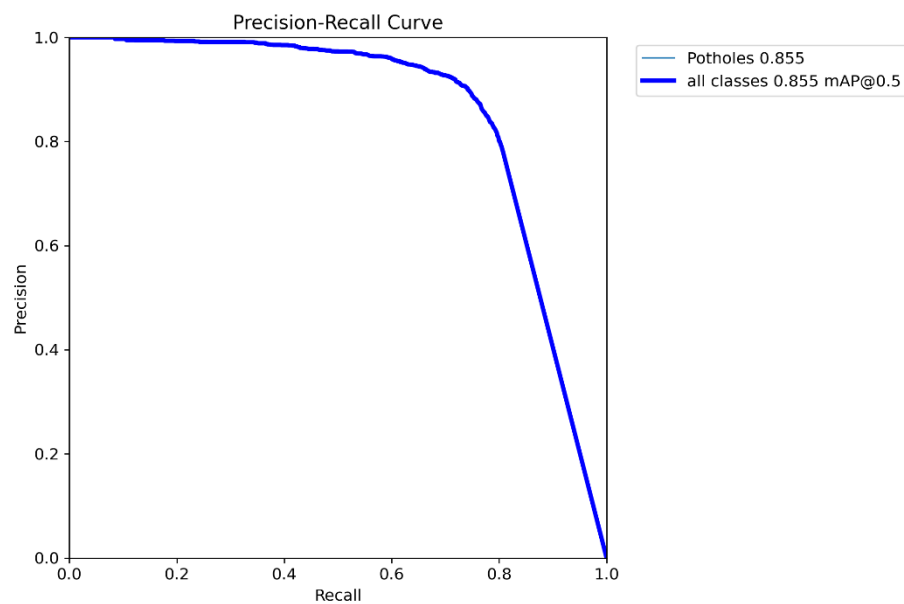


Figure 5.4: Precision-Recall Curve

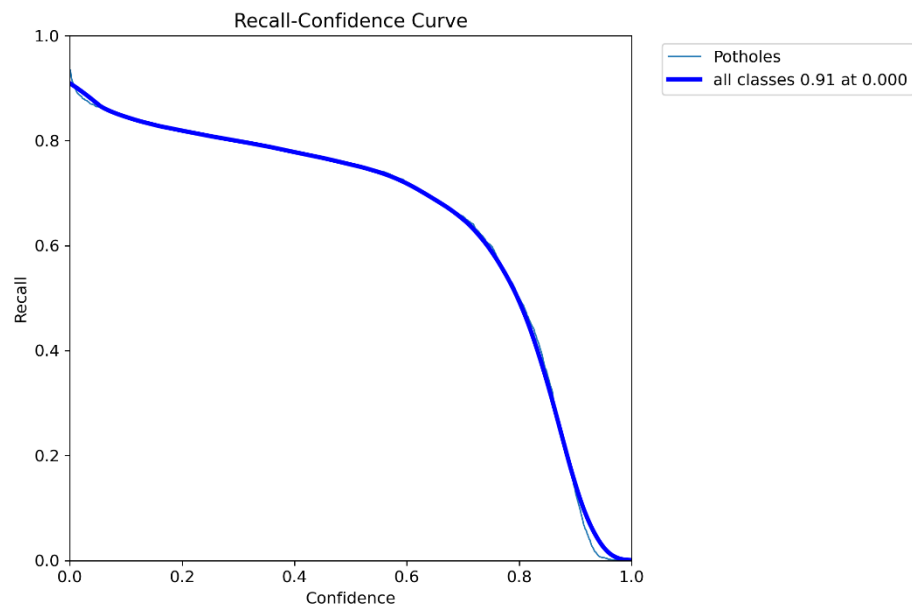


Figure 5.5: Recall-Confidence

Performance Metrics

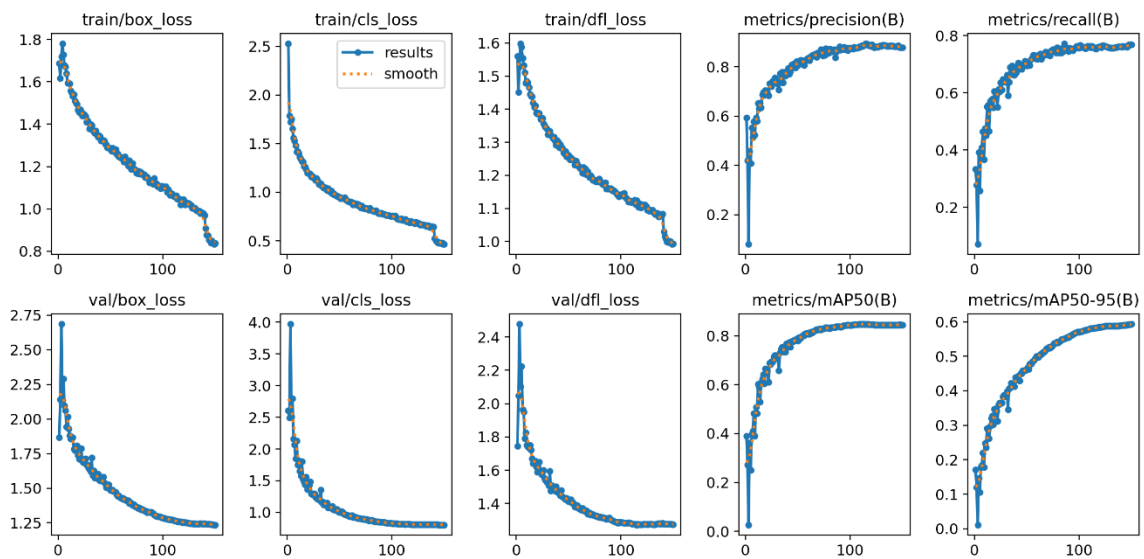


Figure 5.6: Performance metrics

- **Precision**

$$Precision = \frac{TP}{TP + FP} = 85.6\%$$

- **Recall**

$$Recall = \frac{TP}{TP + FN} = 87.7\%$$

- **F1-Score**

$$F1 = 86.6\%$$

- **Mean Average Precision (mAP)**

mAP@0.5: Evaluates detections at IoU = 0.5

mAP@0.5:0.95: Evaluates across multiple IoU thresholds

Table 5.2: mAP metrics

Metric	Value
mAP50	0.71
mAP50-95	0.64



Figure 5.7: Potholes detected by the model

Error Analysis:

Table 5.3: Error Analysis

Error Type	Value
False Positives	279
False Negatives	233

- False positives occur when non-pothole regions are incorrectly detected as potholes.
- False negatives occur when actual potholes are missed by the model.

Model Deployment

The trained YOLOv8n model was converted into ONNX format to enable efficient deployment on embedded systems. The ONNX model was deployed on a Raspberry Pi 5, allowing real-time inference with optimized performance.

Rover-Based System Implementation:

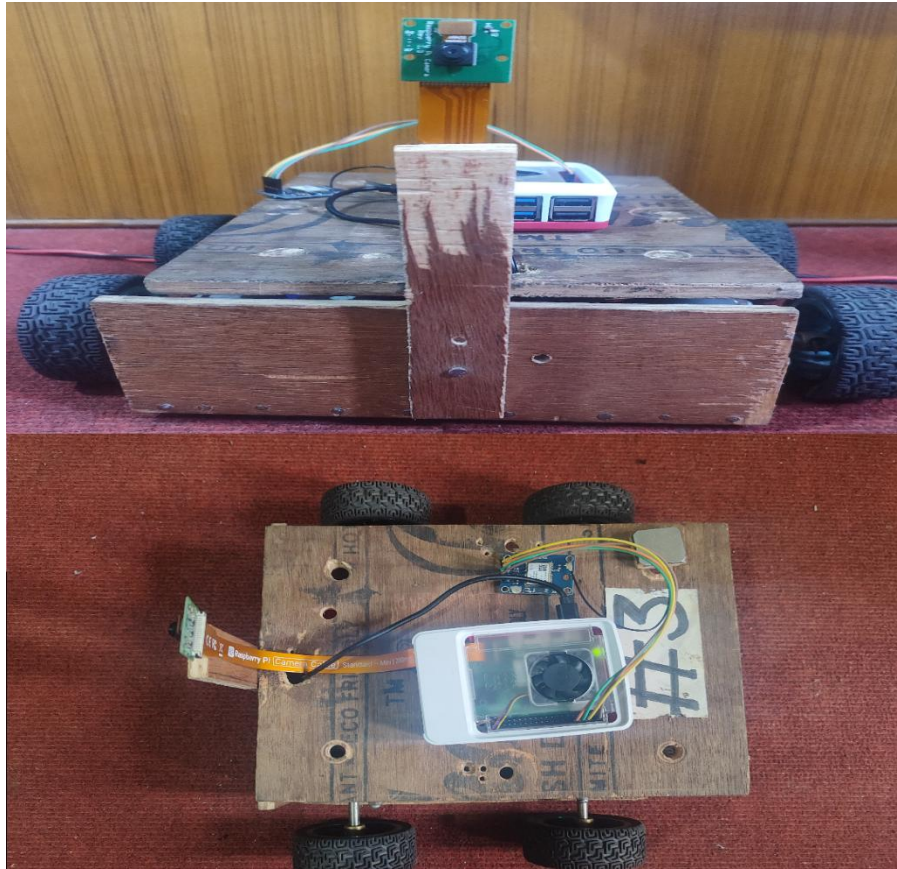


Figure 5.8: Prototype of rover implementation

The rover successfully demonstrated autonomous movement and real-time detection capability in a controlled environment.

Email Alert System:

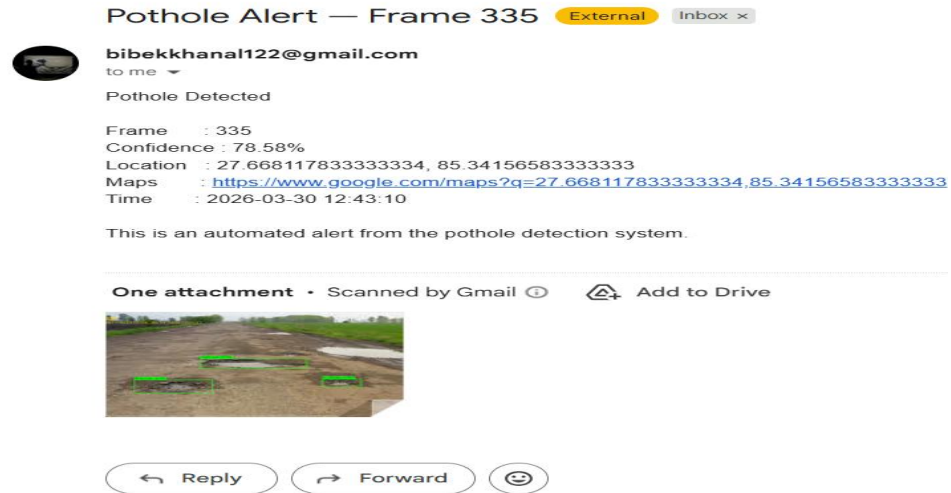


Figure 5.9: Email alert generated by the system

An automated email notification system was integrated into the system.

Real-Time Performance Analysis:

The system was tested under real-world conditions using the rover platform. The following observations were made:

- The model successfully detected potholes in real time
- Detection performance remained consistent on embedded hardware
- Slight latency was observed due to hardware limitations
- The email alert system functioned reliably upon detection

The results demonstrated that the YOLOv8n model performed effectively for pothole detection tasks. The high precision and recall values indicate that the model is capable of accurately detecting potholes while minimizing false detections.

The deployment of the model in ONNX format on Raspberry Pi 5 confirms the feasibility of running deep learning models on edge devices. The rover-based implementation further validates the system's capability to operate in dynamic, real-world environments.

The integration of real-time detection with an automated email alert system enhances the practicality and usability of the solution for smart transportation and infrastructure monitoring.

The project successfully developed a complete pothole detection system using the YOLOv8n model. The system includes dataset preparation, model training, evaluation, deployment, and real-world testing. The model achieved strong performance with high precision, recall, and F1-score, ensuring reliable detection of potholes. The conversion to ONNX format enabled efficient deployment on Raspberry Pi 5, supporting real-time processing.

The rover-based testing demonstrated the system's effectiveness in real-world conditions, while the email alert mechanism provided an additional layer of functionality for remote monitoring and timely response.

Overall, the proposed system is an efficient, scalable, and practical solution for automated pothole detection, contributing to improved road safety and maintenance systems.

6. CONCLUSION

The project titled "Road Condition Detection using YOLOv8 with E-mail Alert System" has successfully met its objectives, showcasing an automated and intelligent approach to road condition monitoring and alert. This study explores the limitations of traditional road inspection methods, introducing a deep learning-based, real-time alternative that is efficient and scalable. The system was designed by incorporating computer vision, embedded systems, and communication technologies, along with training the YOLOv8 model on relevant datasets to detect and classify road surface defects such as potholes and surface degradation. The model demonstrated high accuracy, excellent generalization capabilities, and near real-time execution, making it practical for real-world applications.

Evaluation metrics including precision, recall, and Intersection over Union (IoU) confirmed the detection system's consistency and efficiency across various environmental conditions, lighting, weather, and road types. The hardware implementation utilized Raspberry Pi 5, Pi Camera Module, and NEO-6M GPS module, establishing a compact, cost-effective, and energy-efficient computer system capable of real-time data acquisition and processing. The system autonomously captures road images and processes them with the trained YOLOv8 model without manual intervention, while external storage ensures safe data retention of detection logs and images for later review.

In conclusion, this innovative system not only provides reliable road condition detection and alerts but also sets a foundation for future advancements in cloud integration, real-time dashboards, mobile interfaces, and predictive maintenance derived from data analytics. With further improvements and larger-scale implementation, it has the potential to transform road maintenance practices and enhance the safety and intelligence of transportation infrastructure.

7. LIMITATIONS AND FUTURE WORK

7.1 Limitations

i. Dependence on Image Quality

The system is very dependent on image quality that is captured. It may decrease the accuracy of detection in poor light conditions, shadows, fog, rain, or low-resolution images.

ii. Limited Generalization on Dataset.

The YOLOv8 model is trained with particular data, which means that it might not work equally well with various roads and in particular, those that have other textures, markings, or strange patterns of defects.

iii. Hardware Constraints

The system might be limited by the computational capabilities of a Raspberry Pi 5, interfering with real-time performance counts when dealing with the large-resolution images or with multiple inputs at once.

iv. Addiction to Internet Connection.

The e-mail alerts system must have a stable internet connection. Alerts can take quite some time to arrive or they might not arrive in the remotest places where connection is poor.

7.2 Future Work

i. Creation of a Centralized Dashboard.

It is possible to create a web or mobile based dashboard that will visualise the defects detected, track their position on maps, and monitor the performance of the system in real-time.

ii. Addition of Categories of Detection.

The model may also be re-trained to identify other road related problems like faded lane markings, waterlogging, damage of road signs and debris.

iii. Improved Model Accuracy

The accuracy can be improved by training the model using larger and more diverse datasets, such as images of various areas, weather, and type of roads.

iv. Multi Camera and Wide Area deployment.

A number of cameras can be deployed in various locations and linked to centralized systems to monitor roads on a large scale.

v. Government Systems Integration.

This system can be integrated with municipal or transportation department systems to generate automatically to make tickets and to schedule maintenance.

vi. Offline Alert Mechanism

Local alert systems or SMS based systems could be included so that it could be notified even when there is no net connection.

8. REFERENCES

- [1] A. K. Pandey, "Deep learning approach for road pothole detection using accelerometer and imagery data," Ph.D. dissertation, Coventry University, Coventry, UK, 2021.
- [2] S. S. Gadekar et al., "Real-Time Pothole Detection using Deep Learning," SSRN, 2024. [Online]. Available: <https://ssrn.com/abstract=5086806>
- [3] S. Chandra, K. K. Dubey, G. Agarwal, and N. Chakraborty, "Pothole Detection in Drivable Area using Deep Learning," *Int. Res. J. Multidiscip. Scope (IRJMS)*, vol. 6, no. 1, pp. 349–360, Jan. 2025, doi: 10.47857/irjms.2025.v06i01.02609.
- [4] S. Setty, B. Vasu, D. Manjunath, and M. G., "Detection of Potholes and Road Surface Conditions Using Deep Learning," *J. Emerg. Technol. Innov. Res. (JETIR)*, vol. 10, no. 7, pp. i648–i652, Jul. 2023.
- [5] M. Chanda, S. Rahman, M. T. Ahmed, and M. F. K. Patwary, "Road Recognition and Lane Detection using Deep Learning," *J. Inf. Technol.*, Jul. 2023, doi: 10.59185/dg8xmp38.
- [6] Yao, G.; Zhu, S.; Zhang, L.; Qi, M. HP-YOLOv8: High-Precision Small Object Detection Algorithm for Remote Sensing Images. *Sensors* 2024, 24, 4858. <https://doi.org/10.3390/s24154858>
- [7] Zhao Y, Shi B, Duan X, Zhu W, Ren L, Liao C (2025) Research on road surface damage detection based on SEA-YOLO v8. *PLoS One* 20(6): e0324439. <https://doi.org/10.1371/journal.pone.0324439>

APPENDICES

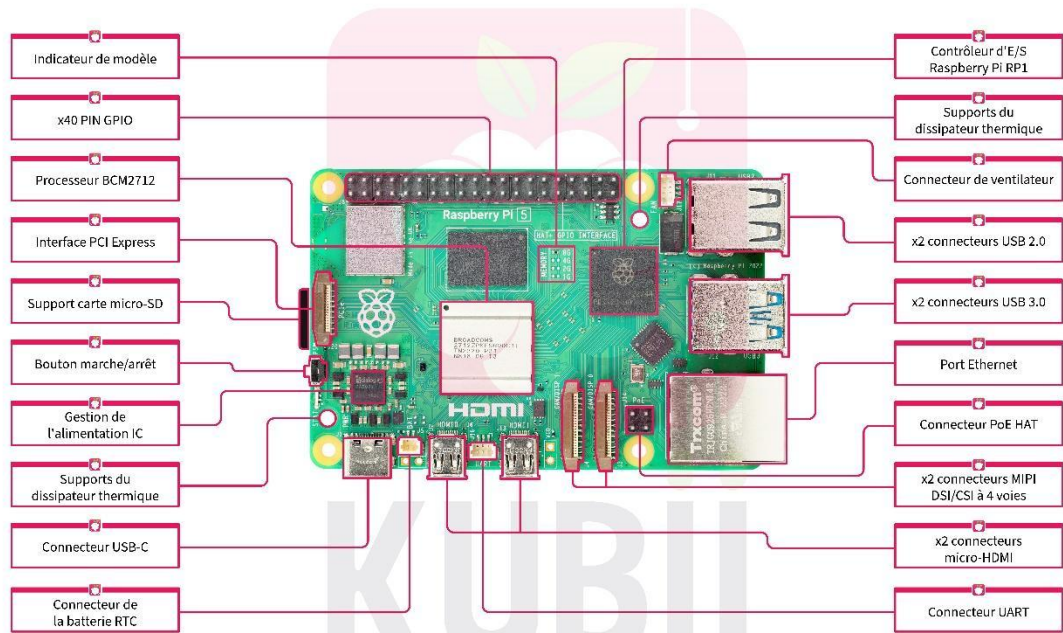


Figure A1: Rasperrypi5 board diagram



Figure A2: NEO-6M GPS module board diagram